COMP301
FINAL REPORT

CAITLIN FOGG
TARIKA SUKDEOA
NASEEHA OSMAN
KERYN SCOTT

INDEX

INSTALLATION AND SETUP GUIDE **3**
ARCHITECTURE **7**
STRUCTURE 7
DESIGN PATTERNS 7
TOOLS AND FUNCTIONALITY 9
AGENT ROLES AND INTERACTION 10
PROMPT ENGINEERING 12
USER INTERFACE 13
ENTROPY REPORT **14**
PROMPTING AND DEBUGGING (HUMAN ENTROPY) 14
MODEL SIZE VS. PERFORMANCE TRADEOFF (SYSTEM ENTROPY) 14
ERROR TRACKING AND FALLBACKS (OPERATIONAL ENTROPY) 14
WHAT THIS MEANS IN THE CONTEXT OF VIBE-CODING 14

INSTALLATION AND SETUP GUIDE

Component	Version/Notes
Python	>= 3.9 (recommended: 3.13)
pip	Comes with Python
LM Studio	Installed locally (for both text and vision models)
Qwen Models (downloaded on LM Studio)	qwen2.5-7b-instruct and qwen2.5-vl-3b-instruct loaded in LM Studio
Internet Connection	Required for accessing OpenFDA & SAHPRA APIs

Guide to the command prompt steps you have to follow to open the program correctly:

```
Microsoft Windows [Version 10.0.26100.6899]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Keryn>D:

D:\>cd D:\University\COMP301\301_Final_Project\301_Final_Project

D:\University\COMP301\301_Final_Project\301_Final_Project>py --version
Python 3.13.5
```

1) Type in the path to the project file (path specific to user and where they have stored the project)
2) Insure you have python installed (must be a version higher than 3.9)

```
D:\University\COMP301\301_Final_Project\301_Final_Project>py -m venv .venv

D:\University\COMP301\301_Final_Project\301_Final_Project>.venv\Scripts\activate

(.venv) D:\University\COMP301\301_Final_Project\301_Final_Project>pip install streamlit
 pandas requests beautifulsoup4 PyPDF2 lxml
```

3) Create the virtual environment
4) Activate it
5) Install Dependencies (even if you think you may have some or all of them installed rather take this step just to be safe). This step only needs to be done the first time you try to access the program.

```
Collecting streamlit
  Using cached streamlit-1.50.0-py3-none-any.whl.metadata (9.5 kB)
Collecting pandas
  Using cached pandas-2.3.3-cp313-cp313-win_amd64.whl.metadata (19 kB)
Collecting requests
  Using cached requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Collecting beautifulsoup4
  Using cached beautifulsoup4-4.14.2-py3-none-any.whl.metadata (3.8 kB)
Collecting PyPDF2
  Using cached pypdf2-3.0.1-py3-none-any.whl.metadata (6.8 kB)
Collecting lxml
  Using cached lxml-6.0.2-cp313-cp313-win_amd64.whl.metadata (3.7 kB)
Collecting altair!=5.4.0,!=5.4.1,<6,>=4.0 (from streamlit)
  Using cached altair-5.5.0-py3-none-any.whl.metadata (11 kB)
Collecting blinker<2,>=1.5.0 (from streamlit)
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting cachetools<7,>=4.0 (from streamlit)
  Using cached cachetools-6.2.1-py3-none-any.whl.metadata (5.5 kB)
Collecting click<9,>=7.0 (from streamlit)
  Using cached click-8.3.0-py3-none-any.whl.metadata (2.6 kB)
Collecting numpy<3,>=1.23 (from streamlit)
  Using cached numpy-2.3.4-cp313-cp313-win_amd64.whl.metadata (60 kB)
Collecting packaging<26,>=20 (from streamlit)
  Using cached packaging-25.0-py3-none-any.whl.metadata (3.3 kB)
Collecting pillow<12,>=7.1.0 (from streamlit)
  Using cached pillow-11.3.0-cp313-cp313-win_amd64.whl.metadata (9.2 kB)
Collecting protobuf<7,>=3.20 (from streamlit)
  Using cached protobuf-6.33.0-cp310-abi3-win_amd64.whl.metadata (593 bytes)
Collecting pyarrow>=7.0 (from streamlit)
  Using cached pyarrow-21.0.0-cp313-cp313-win_amd64.whl.metadata (3.4 kB)
Collecting tenacity<10,>=8.1.0 (from streamlit)
  Using cached tenacity-9.1.2-py3-none-any.whl.metadata (1.2 kB)
Collecting toml<2,>=0.10.1 (from streamlit)
  Using cached toml-0.10.2-py2.py3-none-any.whl.metadata (7.1 kB)
Collecting typing-extensions<5,>=4.4.0 (from streamlit)
```

```
  Using cached typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Collecting watchdog<7,>=2.1.5 (from streamlit)
  Using cached watchdog-6.0.0-py3-none-win_amd64.whl.metadata (44 kB)
Collecting gitpython!=3.1.19,<4,>=3.0.7 (from streamlit)
  Using cached gitpython-3.1.45-py3-none-any.whl.metadata (13 kB)
Collecting pydeck<1,>=0.8.0b4 (from streamlit)
  Using cached pydeck-0.9.1-py2.py3-none-any.whl.metadata (4.1 kB)
Collecting tornado!=6.5.0,<7,>=6.0.3 (from streamlit)
  Using cached tornado-6.5.2-cp39-abi3-win_amd64.whl.metadata (2.9 kB)
Collecting python-dateutil>=2.8.2 (from pandas)
  Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting pytz>=2020.1 (from pandas)
  Using cached pytz-2025.2-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata>=2022.7 (from pandas)
  Using cached tzdata-2025.2-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Using cached charset_normalizer-3.4.4-cp313-cp313-win_amd64.whl.metadata (38 kB)
Collecting idna<4,>=2.5 (from requests)
  Using cached idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Collecting urllib3<3,>=1.21.1 (from requests)
  Using cached urllib3-2.5.0-py3-none-any.whl.metadata (6.5 kB)
Collecting certifi>=2017.4.17 (from requests)
  Using cached certifi-2025.10.5-py3-none-any.whl.metadata (2.5 kB)
Collecting jinja2 (from altair!=5.4.0,!=5.4.1,<6,>=4.0->streamlit)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting jsonschema>=3.0 (from altair!=5.4.0,!=5.4.1,<6,>=4.0->streamlit)
  Using cached jsonschema-4.25.1-py3-none-any.whl.metadata (7.6 kB)
Collecting narwhals>=1.14.2 (from altair!=5.4.0,!=5.4.1,<6,>=4.0->streamlit)
  Using cached narwhals-2.8.0-py3-none-any.whl.metadata (11 kB)
Collecting colorama (from click<9,>=7.0->streamlit)
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting gitdb<5,>=4.0.1 (from gitpython!=3.1.19,<4,>=3.0.7->streamlit)
  Using cached gitdb-4.0.12-py3-none-any.whl.metadata (1.2 kB)
Collecting smmap<6,>=3.0.1 (from gitdb<5,>=4.0.1->gitpython!=3.1.19,<4,>=3.0.7->streaml
it)
```

```
  Using cached smmap-5.0.2-py3-none-any.whl.metadata (4.3 kB)
Collecting soupsieve>1.2 (from beautifulsoup4)
  Using cached soupsieve-2.8-py3-none-any.whl.metadata (4.6 kB)
Collecting MarkupSafe>=2.0 (from jinja2->altair!=5.4.0,!=5.4.1,<6,>=4.0->streamlit)
  Using cached markupsafe-3.0.3-cp313-cp313-win_amd64.whl.metadata (2.8 kB)
Collecting attrs>=22.2.0 (from jsonschema>=3.0->altair!=5.4.0,!=5.4.1,<6,>=4.0->streaml
it)
  Using cached attrs-25.4.0-py3-none-any.whl.metadata (10 kB)
Collecting jsonschema-specifications>=2023.03.6 (from jsonschema>=3.0->altair!=5.4.0,!=
5.4.1,<6,>=4.0->streamlit)
  Using cached jsonschema_specifications-2025.9.1-py3-none-any.whl.metadata (2.9 kB)
Collecting referencing>=0.28.4 (from jsonschema>=3.0->altair!=5.4.0,!=5.4.1,<6,>=4.0->s
treamlit)
  Using cached referencing-0.37.0-py3-none-any.whl.metadata (2.8 kB)
Collecting rpds-py>=0.7.1 (from jsonschema>=3.0->altair!=5.4.0,!=5.4.1,<6,>=4.0->stream
lit)
  Using cached rpds_py-0.27.1-cp313-cp313-win_amd64.whl.metadata (4.3 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.2->pandas)
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Using cached streamlit-1.50.0-py3-none-any.whl (10.1 MB)
Using cached pandas-2.3.3-cp313-cp313-win_amd64.whl (11.0 MB)
Using cached requests-2.32.5-py3-none-any.whl (64 kB)
Using cached altair-5.5.0-py3-none-any.whl (731 kB)
Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
Using cached cachetools-6.2.1-py3-none-any.whl (11 kB)
Using cached charset_normalizer-3.4.4-cp313-cp313-win_amd64.whl (107 kB)
Using cached click-8.3.0-py3-none-any.whl (107 kB)
Using cached gitpython-3.1.45-py3-none-any.whl (208 kB)
Using cached gitdb-4.0.12-py3-none-any.whl (62 kB)
Using cached idna-3.11-py3-none-any.whl (71 kB)
Using cached numpy-2.3.4-cp313-cp313-win_amd64.whl (12.8 MB)
Using cached packaging-25.0-py3-none-any.whl (66 kB)
Using cached pillow-11.3.0-cp313-cp313-win_amd64.whl (7.0 MB)
Using cached protobuf-6.33.0-cp310-abi3-win_amd64.whl (436 kB)
Using cached pydeck-0.9.1-py2.py3-none-any.whl (6.9 MB)
Using cached smmap-5.0.2-py3-none-any.whl (24 kB)
Using cached tenacity-9.1.2-py3-none-any.whl (28 kB)
```

```
Using cached tornado-6.5.2-cp39-abi3-win_amd64.whl (445 kB)
Using cached typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Using cached urllib3-2.5.0-py3-none-any.whl (129 kB)
Using cached watchdog-6.0.0-py3-none-win_amd64.whl (79 kB)
Using cached beautifulsoup4-4.14.2-py3-none-any.whl (106 kB)
Using cached pypdf2-3.0.1-py3-none-any.whl (232 kB)
Using cached lxml-6.0.2-cp313-cp313-win_amd64.whl (4.0 MB)
Using cached certifi-2025.10.5-py3-none-any.whl (163 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached jsonschema-4.25.1-py3-none-any.whl (90 kB)
Using cached attrs-25.4.0-py3-none-any.whl (67 kB)
Using cached jsonschema_specifications-2025.9.1-py3-none-any.whl (18 kB)
Using cached markupsafe-3.0.3-cp313-cp313-win_amd64.whl (15 kB)
Using cached narwhals-2.8.0-py3-none-any.whl (415 kB)
Using cached pyarrow-21.0.0-cp313-cp313-win_amd64.whl (26.1 MB)
Using cached python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Using cached pytz-2025.2-py2.py3-none-any.whl (509 kB)
Using cached referencing-0.37.0-py3-none-any.whl (26 kB)
Using cached rpds_py-0.27.1-cp313-cp313-win_amd64.whl (232 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Using cached soupsieve-2.8-py3-none-any.whl (36 kB)
Using cached tzdata-2025.2-py2.py3-none-any.whl (347 kB)
Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: pytz, watchdog, urllib3, tzdata, typing-extensions, torn
ado, toml, tenacity, soupsieve, smmap, six, rpds-py, PyPDF2, pyarrow, protobuf, pillow,
 packaging, numpy, narwhals, MarkupSafe, lxml, idna, colorama, charset_normalizer, cert
ifi, cachetools, blinker, attrs, requests, referencing, python-dateutil, jinja2, gitdb,
 click, beautifulsoup4, pydeck, pandas, jsonschema-specifications, gitpython, jsonschem
a, altair, streamlit
Successfully installed MarkupSafe-3.0.3 PyPDF2-3.0.1 altair-5.5.0 attrs-25.4.0 beautifu
lsoup4-4.14.2 blinker-1.9.0 cachetools-6.2.1 certifi-2025.10.5 charset_normalizer-3.4.4
 click-8.3.0 colorama-0.4.6 gitdb-4.0.12 gitpython-3.1.45 idna-3.11 jinja2-3.1.6 jsonsc
hema-4.25.1 jsonschema-specifications-2025.9.1 lxml-6.0.2 narwhals-2.8.0 numpy-2.3.4 pa
ckaging-25.0 pandas-2.3.3 pillow-11.3.0 protobuf-6.33.0 pyarrow-21.0.0 pydeck-0.9.1 pyt
hon-dateutil-2.9.0.post0 pytz-2025.2 referencing-0.37.0 requests-2.32.5 rpds-py-0.27.1
six-1.17.0 smmap-5.0.2 soupsieve-2.8 streamlit-1.50.0 tenacity-9.1.2 toml-0.10.2 tornad
o-6.5.2 typing-extensions-4.15.0 tzdata-2025.2 urllib3-2.5.0 watchdog-6.0.0
```

6) Above is all the lines that may appear (especially if you already, like me, have all the packages installed)

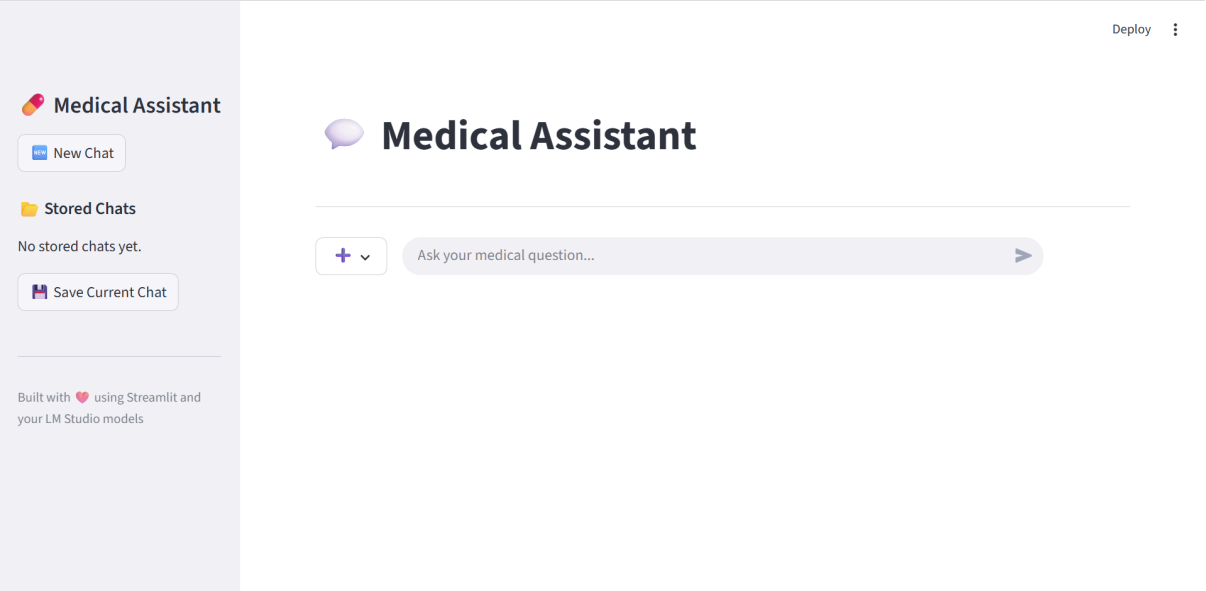
```
[notice] A new release of pip is available: 25.1.1 -> 25.2
[notice] To update, run: python.exe -m pip install --upgrade pip

(.venv) D:\University\COMP301\301_Final_Project\301_Final_Project>streamlit run app.py

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://192.168.1.64:8501
```

7) Type into command prompt “streamlit run app.py”. This will open up the following page on your default browser.



In order to code we have used PyCharm. It is advised that to view the code, one installs and views it in PyCharm.

ARCHITECTURE

STRUCTURE

The medication information assistant is designed as a multi-agent system that integrates image understanding, information retrieval and natural language summarisation to provide accurate and accessible information about any medication identified from an image. The architecture is modular and scalable, ensuring separation of concerns and ease of maintenance.

The system is composed of two main agents and a client interface:

1. Vision Agent (image processing model)
   Responsible for visual recognition and extraction of medication details from uploaded images.

2. Planner Agent (text processing and information integration model)
   Responsible for reasoning, summarisation and formatting of textual information retrieved from multiple sources.

3. Streamlit Front-End (client interface)
   Provides an intuitive interface for users to upload medication images and view summarised results.

These components communicate with each other and work together to form a cohesive system that begins with image input and ends with user-friendly textual output.

DESIGN PATTERNS

The architecture loosely follows the Model–View–Controller (MVC) paradigm:

- Model Layer (Agents and Data Sources):
  Encapsulates data handling and retrieval components such as:

  - Vision Agent
  - Planner Agent.
  - Local Kaggle CSV dataset
  - SAHPRA scraping and PDF parsing
  - OpenFDA API interface
    These represent the system's "knowledge base," forming the foundation for information synthesis.

- Controller Layer (MedicalProcessor):
  The logic that connects the two agents and orchestrates their sequence of operations:
    - The MedicalProcessor class acts as the main controller, with `process_query()` as the controller entry point.
    - It calls the Vision Agent for image analysis, passes the result to the Planner Agent, and coordinates fallback and summarisation logic.
    - The system uses a chain-of-responsibility pattern for source prioritisation — local → SAHPRA → OpenFDA → Vision fallback.
- View Layer (app.py):
  Implemented via the Streamlit web interface, which:
    - Provides a simple upload-and-display workflow for the client.
    - Displays the assistant's summarised output, including drug details, warnings, and source links.
    - Abstracts away all backend complexity, allowing users to interact with the system through a friendly UI.

This clear separation of components ensures that:

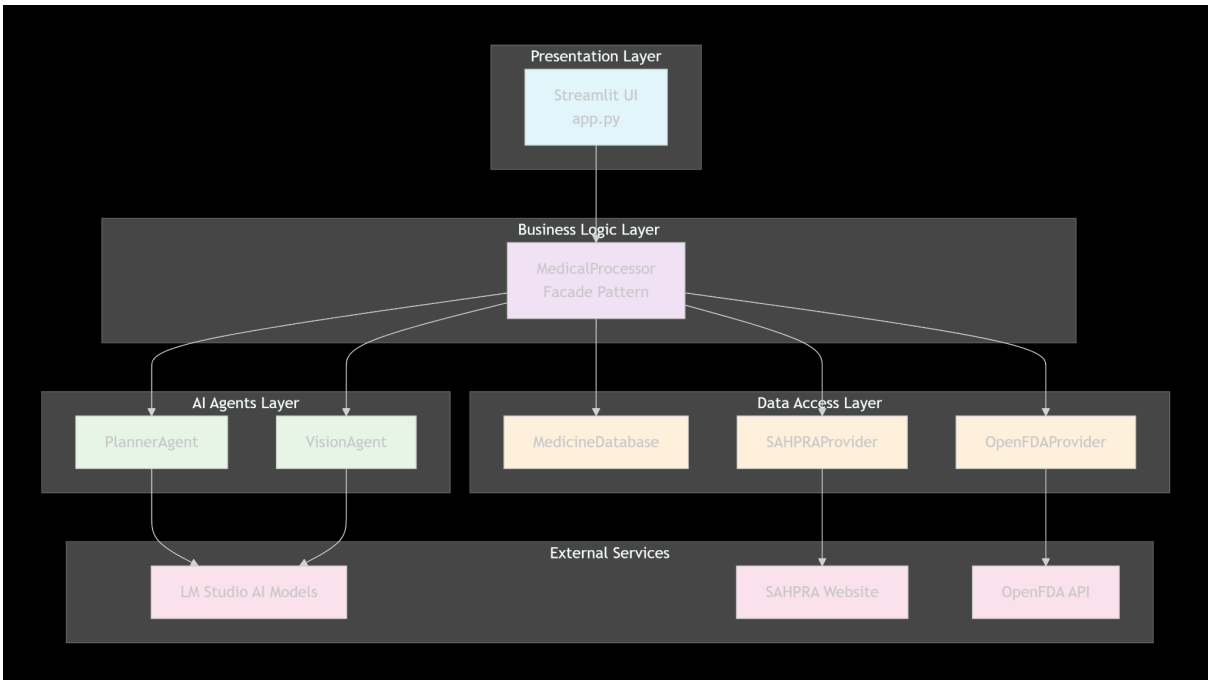
- The Vision Agent operates as an independent module, allowing it to be replaced or upgraded (e.g., switching to a larger vision-language model) without affecting the rest of the system.
- The Planner Agent can incorporate new data sources or summarisation methods.
- The Streamlit UI can be customised without affecting backend logic.

The medical assistant app also utilises the Facade design pattern. This is a structural pattern that provides a simple, unified interface for a complex system of classes, libraries, and frameworks. This hides the complexity of the system whilst providing an entry way to utilise the system. With this design, you only have to communicate with the entry way and not the hidden, more complex systems.

The medical assistant app utilises multiple complex components, making this design suitable. It helps with encapsulation and information hiding. The Streamlit app does not need to know about the vision models, databases, or APIs. The facade handles all the error recovery and the fallback logic. It also makes maintenance and explanation much easier, for example, when adding a new data source, the app.py file remains unchanged.

This is particularly important for medical apps for the following reasons:

- Safety: Centralised error handling and fallback mechanisms
- Reliability: Consistent interface even when some data sources fail
- Maintainability: Easy to update medical data sources without affecting the UI
- Scalability: Simple to add new AI models or medical databases



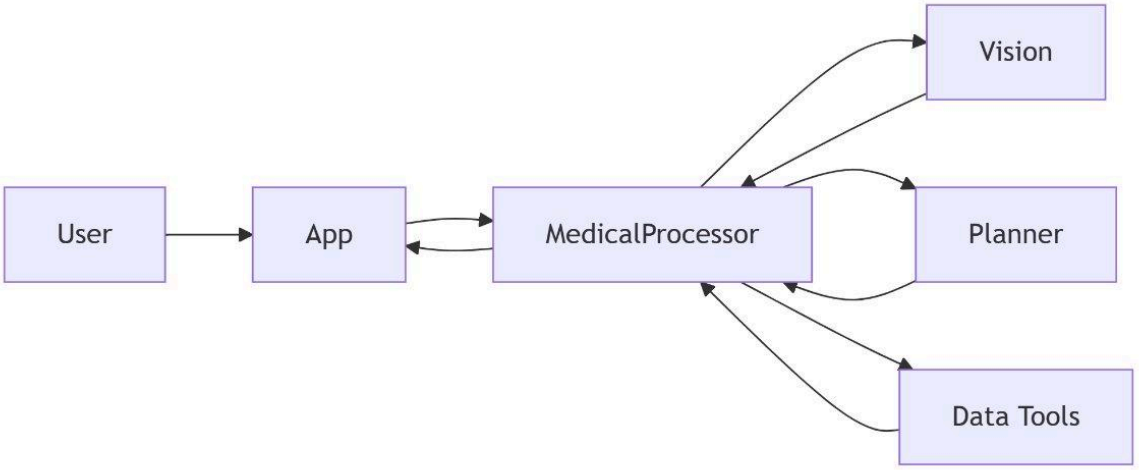
Above is a diagram of the architecture

TOOLS AND FUNCTIONALITY

Data Tools

- Database Query Tool:
  - Uses local CSV medication dataset (from kaggle)
  - Implements fuzzy name matching to handle types and variations
- SAHPRA Web Scraping Tool
  - Extracts official South African medicine information
  - Implements PDF parsing
- OpenFDA API Tool
  - Global regulatory drug label information

AGENT ROLES AND INTERACTION



Vision Agent

Model used: `qwen2.5-vl-3b-instruct` from LMStudio

Purpose: To analyse the image of the medication packaging (bottle, box or blister pack) and extract key textual details such as the drug name, dosage strength and manufacturer name.

Workflow:

1. When a user uploads an image, the Vision Agent encodes it into base64 and sends it to the multimodal model endpoint.

2. The model processes both image and prompt text to extract structured information, returning results in JSON format.

3. A parsing layer cleans and validates the model's output, ensuring it can be easily passed to the next stage.

Planner Agent

Model used: `qwen2.5-7b-instruct` from LMStudio

Purpose: To handle all reasoning, summarisation and text synthesis tasks once the medication has been identified.

Core responsibilities:

1. Query and retrieve data about the identified drug from multiple knowledge sources:

    - Local Kaggle dataset (offline structured drug information)
    - SAHPRA repository (official South African medicine information PDFs)
    - OpenFDA API (global regulatory drug label information)

2. Extract, clean, and format relevant sections such as uses/indications, dosage and administration, side effects and adverse reactions, and warnings and contraindications

3. Summarise this data into a concise, standardised JSON structure for easy display to the user.

How they work together to produce results

The web supplementation function in the MedicalProcessor class ensures that the assistant can still provide complete and reliable medication information even when local data sources are insufficient or unavailable. It is implemented primarily through the _safe_database_search() method, which coordinates a multi-layered retrieval and summarisation pipeline combining local, national and international data sources.

When a medication name is identified (either from image extraction from the Vision Agent or user text input), the system first attempts to locate the drug within the local Kaggle-based medicine database. If no reliable match or complete information is found, it sequentially queries external web sources to supplement the missing data:

1. SAHPRA Provider (South African Health Products Regulatory Authority): Accesses locally relevant regulatory documents and medical information for drugs approved in South Africa.

2. OpenFDA Provider: Fetches U.S. FDA-approved medication labels and descriptions to ensure international coverage and authoritative accuracy.

Each source's raw text or document content is passed to the Planner Agent, which uses natural-language prompting to extract and summarise key details (name, dosage, uses, side effects and warnings) into structured JSON output.

If all web and local lookups fail or return incomplete results, the system automatically invokes a vision-language (VL) model fallback that generates a minimal summary directly from the image analysis output. This fallback ensures robustness and continuity in the assistant's performance.

PROMPT ENGINEERING

Vision Agent prompt:

```
(
    "Look at the provided image of a medication package or blister pack. "
    "Extract the drug name (brand or generic), the dosage strength (e.g., 500 mg), "
    "and any manufacturer name you can find. Return ONLY valid JSON with keys:
drug_name, dosage, manufacturer."
)
```

Provides structured output to pass to Planner agent

Planner Agent prompt: (data from data tools)

```
(
        f"You are a medical info assistant. Given the following raw authoritative text for
'{drug_name}', "
        "please extract and summarize into clear bullet points with these fields: "
        "Name, Uses/Indications, Typical Dosage (if present), Common Side Effects,
Warnings/Interactions, Source.\n\n"
        "Raw text:\n" + source_text +
        "\n\nReturn ONLY valid JSON with keys: name, uses, dosage, side_effects, warnings,
source. "
        "DO NOT add any text before or after the JSON object."
)
```

This prompt ensures that only consistent medical information is extracted and is in correct json formatting.

Prompt for general query Planner Agent:

```
(
    f"You are a helpful and reliable medical assistant.\n"
    f"Answer the following user question clearly and accurately:\n\n"
    f"{query}\n\n"
    "If it refers to a drug or medication, include details on its uses, dosage, side effects, "
    "warnings, and interactions if relevant. Provide a structured response that can be easily
displayed."
)
```

Ensures special reasoning and domain specific responses. By using role based context conditioning we ensure reliable, specific and helpful responses.

USER INTERFACE

The Streamlit frontend acts as the user interaction layer. It enables clients to:

- Upload an image of a medication directly from their device.
- Call on the agents and their respective functions to process the image and perform information retrieval.
- View neatly formatted results including the drug name and dosage, indications and usage, side effects and cautions, external source references (e.g., SAHPRA or OpenFDA links)

ENTROPY REPORT

Entropy was quantified by continuously logging, timing, and analysing both human-AI interaction and system-level performance to find the most stable configuration of the AI-driven development pipeline.

PROMPTING AND DEBUGGING (HUMAN ENTROPY)

Considerable time was spent iteratively prompting AI to fine-tune the code, testing outputs, and addressing inconsistent or incomplete responses. Each round of prompting and correction reduced system uncertainty but at the cost of additional human effort.

This was tracked through prompt logs with the AI and printing output at each stage of the code execution, which included results of the medicine search and fallback and error messages, such as model or search failure, to track whether the dataset search and web supplementation functions were retrieving complete and correct results, or if another round of correction was needed.

Debugging Complexity:

- Low: Image processing worked reliably
- Medium: JSON parsing between agents required refinement
- High: Web scraping accuracy needed multiple iterations

MODEL SIZE VS. PERFORMANCE TRADEOFF (SYSTEM ENTROPY)

Initially, various larger models that either exceeded hardware capacity or performed too slowly were experimented with. Through iterative testing (printing intermediate outputs, timing execution steps, and monitoring task completion) a balanced configuration was reached, making use of lighter weight models that still maintained good speed and accuracy.

ERROR TRACKING AND FALLBACKS (OPERATIONAL ENTROPY)

As mentioned above, runtime errors and fallbacks were logged to observe how often the system deviated from expected behaviour. All output was compared with the information in the local dataset and the PDFs from the SAPHRA website, and if the results of the assistant's search were incomplete or incorrect, more fine-tuning (for example refining regular expression matches) was applied.

WHAT THIS MEANS IN THE CONTEXT OF VIBE-CODING

This entropy cost can be generalised to reflect the limitations of vibe coding in any context: While AI-driven development can significantly improve efficiency, adaptability and innovation speed, it also introduces new layers of complexity that can only be addressed by human

effort and oversight. When developers are orchestrating rather than authoring code, there will always be a need for testing and fine-tuning the software and its results. AI can also misinterpret data, produce incomplete results, or fail to account for important context-specific variables (e.g. patients' personal needs). Therefore, qualified software engineers and other industry professionals must remain central to the process - ensuring that outputs are accurate, safe and ethically sound no matter how reliable the system may seem.

Although the entropy cost of human-AI collaboration will vary depending on what context it is used in, as models evolve, the combination of human expertise and AI capability will enable more intelligent and trustworthy systems. This will help lower the entropy cost of AI-assisted development, and promises a collaborative and lucrative environment where technology acts as a support for, rather than a replacement of, professionals across disciplines.